

Mission Geometry:
Planning, Conformance, and Separability
for Bounded Receipted Execution

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Pillar III

Abstract

This is Pillar III of the Chatman Equation thesis series: the geometry beneath the manufacturing morphism μ and the conformance invariant of the enforced equation. We treat mission execution as two dual questions — *manufacture as search* (how a plan is produced) and *conformance as geometry* (how a trace is admitted) — and give both a first-principles account grounded line-for-line in the `praxis` and `bcinr-pddl` codebase. We define the PDDL action law with numeric fluents, and derive the attention fluent balance law as the feasibility condition of a durative-action schedule, with conservation of the borrowed-and-returned fluent proved directly. We build the marking polytope from the Petri state equation $\mathbf{m} = \mathbf{m}_0 + \mathbf{N}\mathbf{x}$, show the STRIPS grounding is a place/transition net whose forward search inhabits marking space, and prove that the lifecycle token verifier’s branchless firing rule $\text{enabled} \leftarrow (\text{enabled} \wedge \neg \mathbf{m}^-) \vee \mathbf{m}^+$ is exactly the safe-net specialization of that equation. We formalize conformance as membership in the reachability cone, and prove a Farkas separation theorem: a nonconformant marking admits a separating-hyperplane certificate of dimension p , sound for non-reachability and verifiable at $O(\kappa)$. We define token-replay fitness as the population ratio the `PowlReplayVerifier` computes in Q16.16, and prove that unit fitness with completion characterizes exactly the genuine firing sequences, with the first disabled transition a coordinate-localized witness. We formalize POWL 2.0 as recursive partial orders and choice graphs, show the plan-to-tape transitive reduction yields the Hasse diagram of the precedence order, and reframe the Kourani–van der Aalst separable decomposition [6] as a Rice quarantine for processes: separable is admitted, non-separable is refused with reason. Finally we derive lexicographic cost arbitration as the infinite-weight limit in which the admitted coordinate dominates, and give the makespan functional with its critical-path and capacity-sensitivity structure as computed by `analyze_schedule`. Every theorem is anchored to the `bcinr-pddl` planner (`find_plan`, `find_temporal_plan`), the `PowlReplayVerifier`, the capability router `CostVector`, and the `ScheduleAnalysis64` analyzer.

Contents

Abstract	i
1 Introduction: Manufacture as Search, Conformance as Geometry	1
1.1 Where this pillar sits	1
1.2 The two theses of this pillar, stated once	1
2 The PDDL Action Law with Numeric Fluents	3
2.1 Lifted domains, grounding, and the state	3
2.2 Numeric fluents and the attention balance law	4
3 The Marking Polytope and the State Equation	6
3.1 Place/transition nets and the incidence matrix	6
3.2 STRIPS grounding is a net; the lifecycle token model is its safe specialization .	7
4 Conformance as Cone Membership and Farkas Certificates	8
4.1 Conformance is membership	8
5 Token-Replay Fitness as Normalized Distance	10
5.1 The replay verifier as a Petri semantics	10
6 POWL 2.0: Choice Graphs and Partial Orders	12
6.1 The language	12
7 Separability as a Rice Quarantine for Processes	14
7.1 The separable decomposition	14
8 Cost Arbitration and the Makespan Functional	16
8.1 The cost vector and lexicographic order	16
8.2 The makespan functional and critical-path structure	17
9 Conclusion	19
A Notation	20
B Code Correspondence	21

Chapter 1

Introduction: Manufacture as Search, Conformance as Geometry

1.1 Where this pillar sits

The foundations paper of this series fixed the Chatman equation $\mathcal{A} = \mu(\mathcal{O}^*)$ and its enforced form, the Bounded Receipted Chatman Equation (BRCE there), a conjunction of four invariants: an admission gate, a bounded deterministic manufacture, receipt totality, and *conformance* — the requirement that an actuation’s lifecycle replay have fitness $\varphi = 1$. Two of those invariants are the subject of this pillar. The manufacturing morphism μ , when its artifact is a *mission* — a schedule of durative actions achieving a goal — is a bounded search over a state space. The conformance invariant is a *geometric* membership test: does a trace lie inside the polytope of lawful markings? We develop both from first principles and show they are the same object viewed from two sides.

Question	Object	praxis/bcinr-pddl realization
Manufacture (μ)	bounded search	<code>find_plan</code> (BFS), <code>find_temporal_plan</code>
Conformance (B4)	cone membership	<code>PowlReplayVerifier</code> , marking geometry
Arbitration	lexicographic order	<code>CostVector</code> , <code>route_capability_plan</code>
Makespan / cost	critical-path functional	<code>ScheduleAnalysis64</code>

1.2 The two theses of this pillar, stated once

Manufacture is bounded search over a finite ground net. A lifted PDDL domain with numeric fluents grounds to a *finite* set of transitions (Theorem 2.1), so μ ’s search terminates with a priori bounded cost — the boundedness law (M2) of the foundations, instantiated. The state it searches is a marking: a vector of atom-tokens plus a valuation of numeric fluents. The attention fluent, borrowed at an action’s start and returned at its end, obeys a balance law whose nonnegativity is exactly schedule feasibility (Proposition 2.3).

Conformance is distance to a cone, certified by a hyperplane. The reachable markings of a net are constrained by the state equation $\mathbf{m} = \mathbf{m}_0 + \mathbf{N}\mathbf{x}$ to lie in a translated cone (Proposition 3.1). A trace conforms only if its marking lies there; a nonconformant marking is

separated from the cone by a Farkas hyperplane (Theorem 4.1), a certificate of dimension p that a bounded verifier checks without replaying the interior. Token-replay fitness measures the normalized excursion outside the enabled set, and equals 1 exactly on genuine firing sequences (Theorem 5.1).

Throughout, a claim is a definition, a theorem with proof, or a citation. The prior published paper [1] demonstrated $\mathcal{A} = \mu(\mathcal{O})$ at enterprise scale and full coverage of the forty-three workflow patterns [8]; this pillar supplies the planning and conformance geometry those numbers rested on.

Chapter 2

The PDDL Action Law with Numeric Fluents

2.1 Lifted domains, grounding, and the state

We take the numeric-temporal fragment of PDDL 2.1 [2] as the language of μ when it manufactures missions, and give it exactly the structure `bcinr-pddl` grounds and searches.

Definition 2.1 (Lifted numeric-temporal domain). A *lifted domain* is a tuple $\mathcal{D} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{S})$: a finite type hierarchy $\mathcal{T}$ (a forest under a subtype relation, with universal root `object`); a finite set $\mathcal{P}$ of predicate symbols with typed arities; a finite set $\mathcal{F}$ of numeric *function* symbols (fluents) with typed arities; and a finite set $\mathcal{S}$ of action schemas. A *durative* schema $s \in \mathcal{S}$ carries typed parameters $\bar{v}$, a duration constraint, a condition C_s , and an effect E_s , where conditions and effects are timestamped `at start` / `at end` and may be atomic (over $\mathcal{P}$) or numeric comparisons/updates (over $\mathcal{F}$).

Definition 2.2 (Grounded temporal problem). Given a domain $\mathcal{D}$ and a problem $(\mathcal{O}, \mathbf{m}_0, \nu_0, \gamma)$ with finite typed objects $\mathcal{O}$, initial atom set $\mathbf{m}_0$, initial fluent valuation $\nu_0 : \mathcal{F}\text{-instances} \rightarrow \mathbb{R}$, and goal condition γ , the *grounding* substitutes every type-compatible object tuple into each schema’s parameters, producing a finite set of ground actions. A grounded state is a pair $(\mathbf{m}, \nu)$ of a set $\mathbf{m}$ of true ground atoms and a fluent valuation ν .

This is the `GroundTemporalProblem` of `bcinr-pddl`: `initial_atoms`, `initial_fn_values`, `grounded_durative_actions`, and a goal `PddlCondition`. Type compatibility is the `TypeIndex::satisfies` walk up the parent chain; the universal type `object` matches every object.

Theorem 2.1 (Grounding is bounded). *Let $\mathcal{D}$ have schemas of maximum parameter arity k over $|\mathcal{O}| = N$ objects. The number of ground actions is at most $|\mathcal{S}| \cdot N^k$, a finite constant independent of the goal. Consequently a forward search over grounded actions has a branching factor bounded a priori, and μ ’s manufacture terminates within a fixed depth bound.*

Proof. Each schema binds at most k parameters, each to one of $\leq N$ objects, so it grounds to at most N^k actions; summing over $|\mathcal{S}|$ schemas gives the bound. In `bcinr-pddl` the count is additionally capped by the structural constant `PDDL8_MAX_GROUND`: exceeding it is the hard error `Pddl8Error::BoundExceeded`, and an empty grounding is `EmptyGrounding`. Thus the ground action set is finite and its size is a fixed function of $(|\mathcal{S}|, N, k)$, discharging the boundedness law (M2) for mission manufacture. $\square$

Remark (The forward search is the manufacture). For the classical (atomic) fragment, `GroundProblem::find_plan` is breadth-first search over sets of ground atoms: the frontier is a queue of (state, path) pairs, visited states are deduplicated, and the first state containing the goal set returns its path as a `Pddl8Tape`. Search failure within `PDDL8_MAX_PLAN_DEPTH` is `NoAdmittedPlan` — a *refusal with reason*, not an exception, exactly as admission returns $\perp$. The manufacture either yields an artifact or a receipted refusal; it never diverges, by Theorem 2.1.

2.2 Numeric fluents and the attention balance law

The distinctive content of the numeric fragment is that an action may test and update a real-valued fluent. `bcinr-pddl`’s `eval_numeric` evaluates a fluent expression against a valuation, and `apply_numeric_effect` realizes the five update kinds `Assign`, `Increase`, `Decrease`, `ScaleUp`, `ScaleDown`. We formalize the one that carries the calculus of attention.

Definition 2.3 (Numeric fluent balance). A durative action j *draws* on a fluent ϕ if its effect contains **at start** (**decrease** ϕ r_j) and **at end** (**increase** ϕ r_j) for a rate $r_j \geq 0$, and its condition contains **at start** ($\phi \geq r_j$). Over the half-open interval $[s_j, e_j)$ of its execution, j holds r_j units of ϕ and releases them at e_j . The *free level* of ϕ at time t is

$$f_\phi(t) = \nu_0(\phi) - \sum_{j: t \in [s_j, e_j)} r_j.$$

This is exactly the capability router’s model: its domain declares `(:functions (attention))`; each capability schema decrements `attention` by 1 **at start** under the guard `(>= (attention) 1)` and increments it back **at end**. Attention is “one working human, one active train of thought.”

Proposition 2.2 (Attention is borrowed and returned: conservation). *If every action drawing on ϕ both decrements ϕ by r_j at start and increments it by r_j at end, then the net effect of each completed action on ϕ is zero, and the terminal valuation equals the initial: $\nu_{\text{end}}(\phi) = \nu_0(\phi)$. Hence ϕ is a conserved resource; scheduling redistributes when it is held, never how much exists.*

Proof. By `apply_numeric_effect`, `Decrease`(ϕ, r_j) then `Increase`(ϕ, r_j) compose to the identity on $\nu(\phi)$. Summing over all completed actions, the total change is $\sum_j (r_j - r_j) = 0$, so $\nu_{\text{end}}(\phi) = \nu_0(\phi)$. The intermediate values differ only on the union of the open execution intervals, which is the content of Definition 2.3. $\square$

Proposition 2.3 (Feasibility is pointwise nonnegativity). *A durative-action schedule is feasible with respect to ϕ — every action’s start guard ($\phi \geq r_j$) holds when it fires — if and only if $f_\phi(t) \geq 0$ for all t ; equivalently, at no instant does the total rate of in-flight draws exceed $\nu_0(\phi)$. For the unit-rate attention fluent ($r_j \equiv 1$), this says the number of concurrently in-flight capabilities never exceeds the capacity $\nu_0(\text{attention})$.*

Proof. The start guard at s_j requires $\nu(\phi) \geq r_j$ in the state just before j ’s at-start decrement. By Definition 2.3, that state’s value of ϕ is $f_\phi(s_j^-) = \nu_0(\phi) - \sum_{i \neq j: s_j \in [s_i, e_i)} r_i$, so the guard holds iff $f_\phi(s_j^-) \geq r_j$, i.e. $f_\phi(s_j) \geq 0$ after j ’s own draw is included. Free level changes only at action starts and ends, so if it is ≥ 0 at every start it is ≥ 0 everywhere. Conversely a violated

guard is an instant with $f_\phi < 0$. For $r_j \equiv 1$, the sum counts in-flight actions, so nonnegativity is exactly “concurrency $\leq$ capacity.” $\square$

`bcinr-pddl`’s temporal planner enforces exactly this: within a tick it re-scans candidate actions after each start so an `at start` decrement is visible to the next candidate’s guard in the same instant, which the source comments identify as “what lets concurrent starts correctly gate on shared resource fluents.” The `zero_attention_capacity_is_infeasible_and_refused` test is Proposition 2.3 at capacity 0: with $f_{\text{attention}}(t) < 0$ forced at any start, the router refuses rather than defaults a route.

Chapter 3

The Marking Polytope and the State Equation

3.1 Place/transition nets and the incidence matrix

Atomic (non-numeric) execution has a native linear-algebraic geometry, that of place/transition nets [3]. We reproduce it in the form the code inhabits.

Definition 3.1 (Net, marking, enabling, firing). A *net* has p places and a finite set T of transitions. A *marking* is a vector $\mathbf{m} \in \mathbb{Z}_{\geq 0}^p$. A transition t is a pair $(\mathbf{m}_t^-, \mathbf{m}_t^+)$ of a *preset* (consumed) and *postset* (produced) vector. t is *enabled* at $\mathbf{m}$ iff $\mathbf{m} \geq \mathbf{m}_t^-$ coordinatewise; firing it yields $\mathbf{m}' = \mathbf{m} - \mathbf{m}_t^- + \mathbf{m}_t^+$.

Definition 3.2 (Incidence matrix and state equation). Let $\mathbf{N} \in \mathbb{Z}^{p \times |T|}$ have column $\delta_t = \mathbf{m}_t^+ - \mathbf{m}_t^-$ for each transition t . A firing sequence ς with *Parikh vector* $\mathbf{x} \in \mathbb{Z}_{\geq 0}^{|T|}$ (the count of each transition's firings) satisfies the *state equation*

$$\mathbf{m} = \mathbf{m}_0 + \mathbf{N}\mathbf{x}. \quad (3.1)$$

Proposition 3.1 (Reachability lies in a translated cone). *Every marking $\mathbf{m}$ reachable from $\mathbf{m}_0$ satisfies (3.1) for some $\mathbf{x} \in \mathbb{Z}_{\geq 0}^{|T|}$, hence lies in the integer points of $\mathbf{m}_0 + \text{cone}(\mathbf{N})$ intersected with $\mathbb{Z}_{\geq 0}^p$, where $\text{cone}(\mathbf{N}) = \{\mathbf{N}\mathbf{x} : \mathbf{x} \geq \mathbf{0}\}$. The state equation is a necessary condition for reachability; it is not sufficient (a $\mathbf{x}$ solving (3.1) need not correspond to a fireable ordering).*

Proof. Firing t once adds column δ_t to the marking (Definition 3.1); a sequence firing each t exactly x_t times adds $\sum_t x_t \delta_t = \mathbf{N}\mathbf{x}$, giving (3.1). Since each $x_t \geq 0$ and markings are nonnegative integers, $\mathbf{m}$ is an integer point of $\mathbf{m}_0 + \text{cone}(\mathbf{N})$ in the nonnegative orthant. Insufficiency is the classical Petri-net fact [3]: (3.1) ignores intermediate nonnegativity and firing order, so spurious solutions exist. $\square$

Definition 3.3 (Marking polytope). The *marking polytope* of a net is the relaxation

$$\mathcal{P} = \{ \mathbf{m}_0 + \mathbf{N}\mathbf{x} : \mathbf{x} \geq \mathbf{0} \} \cap \{ \mathbf{m} \geq \mathbf{0} \} \subseteq \mathbb{R}_{\geq 0}^p,$$

the continuous translated cone whose integer points over-approximate the reachable set.

3.2 STRIPS grounding is a net; the lifecycle token model is its safe specialization

Construction 3.1 (STRIPS grounding as a net). Take the places to be the ground atoms. A ground action t with delete-effects $\mathbf{m}_t^-$ and add-effects $\mathbf{m}_t^+$ is a transition; its precondition atoms must all be present for it to fire (an enabling condition refining Definition 3.1). Then `GroundProblem::find_plan` searches marking space: the BFS frontier is a set of markings, the successor relation is exactly transition firing (`for d in del_effects { next.remove(d) }`; `for a in add_effects { next.insert(a) }`), and a plan is a path to a marking containing the goal.

The lifecycle model of the foundations paper — the fixed sequence `judge` $\rightarrow$ `admit` $\rightarrow$ `receipt` — is a concrete net whose places are the token bits of `replay_adapter`: `TOK_START`, `TOK_JUDGED`, `TOK_ADMITTED`, `TOK_DONE`. Each step’s `required_tokens` is its preset and `produces_tokens` its postset:

`judge` : `TOK_START` $\rightarrow$ `TOK_JUDGED`, `admit` : `TOK_JUDGED` $\rightarrow$ `TOK_ADMITTED`, `receipt` : `TOK_ADMITTED` $\rightarrow$ `TOK_DONE`

Proposition 3.2 (The verifier’s firing rule is the safe-net state equation). *On a safe (1-bounded) net, where every marking and every preset/postset is a 0/1 vector and presets are consumed, the integer firing rule $\mathbf{m}' = \mathbf{m} - \mathbf{m}_t^- + \mathbf{m}_t^+$ of Definition 3.1 coincides with the branchless bitset update the `PowlReplayVerifier` performs:*

$$\text{enabled_tokens} \leftarrow (\text{enabled_tokens} \ \& \ \neg \mathbf{m}_t^-) \mid \mathbf{m}_t^+,$$

and the enabling test $\mathbf{m} \geq \mathbf{m}_t^-$ coincides with the branchless subset check $(\text{enabled} \ \& \ \mathbf{m}_t^-) \oplus \mathbf{m}_t^- = 0$.

Proof. On 0/1 markings with $\mathbf{m}_t^- \leq \mathbf{m}$, the AND-NOT `enabled` $\& \neg \mathbf{m}_t^-$ clears exactly the consumed places (subtracting $\mathbf{m}_t^-$), and the OR with $\mathbf{m}_t^+$ sets the produced places (adding $\mathbf{m}_t^+$); when preset and postset are disjoint from the retained marking this is bit-for-bit the integer expression. The enabling identity holds because $(\mathbf{m} \ \& \ \mathbf{m}_t^-) \oplus \mathbf{m}_t^- = 0$ iff every bit of $\mathbf{m}_t^-$ is also set in $\mathbf{m}$, i.e. $\mathbf{m} \geq \mathbf{m}_t^-$ coordinatewise on 0/1 vectors. Both are the verifier’s two guard lines verbatim. $\square$

Proposition 3.2 is the bridge between the general Petri geometry of [3] and the constant-time bitwise mechanism the receipt path actually runs: the polytope theory is the semantics, the mask reduction is the implementation.

Chapter 4

Conformance as Cone Membership and Farkas Certificates

4.1 Conformance is membership

Definition 4.1 (Conformance). A trace with final marking $\mathbf{m}^\dagger$ and transition-count vector $\mathbf{x}$ *conforms* to a net with initial marking $\mathbf{m}_0$ if $\mathbf{m}^\dagger \in \mathcal{P}$ and the $\mathbf{x}$ witnessing it is realized by a genuine firing ordering. A trace *fails membership* if $\mathbf{m}^\dagger \notin \mathcal{P}$, i.e. no $\mathbf{x} \geq \mathbf{0}$ solves $\mathbf{N}\mathbf{x} = \mathbf{m}^\dagger - \mathbf{m}_0$ within the nonnegative orthant.

Theorem 4.1 (Farkas separation certifies nonconformance). *Fix $\mathbf{b} = \mathbf{m}^\dagger - \mathbf{m}_0$. Exactly one of the following holds:*

- (a) *there exists $\mathbf{x} \geq \mathbf{0}$ with $\mathbf{N}\mathbf{x} = \mathbf{b}$ (the continuous relaxation is feasible; membership in the cone is possible); or*
- (b) *there exists a vector $\mathbf{y} \in \mathbb{R}^p$ with $\mathbf{N}^\top \mathbf{y} \leq \mathbf{0}$ and $\mathbf{y}^\top \mathbf{b} > 0$ — a separating hyperplane whose normal $\mathbf{y}$ certifies $\mathbf{m}^\dagger \notin \mathbf{m}_0 + \text{cone}(\mathbf{N})$.*

The certificate $\mathbf{y}$ is a sound proof of nonconformance: if (b) holds, no firing sequence, in any order, reaches $\mathbf{m}^\dagger$ from $\mathbf{m}_0$.

Proof. This is Farkas’ lemma [4] applied to the system $\mathbf{N}\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$: either the system has a nonnegative solution, or there is $\mathbf{y}$ with $\mathbf{y}^\top \mathbf{N} \leq \mathbf{0}^\top$ and $\mathbf{y}^\top \mathbf{b} > 0$, and never both. In case (b), suppose for contradiction a firing sequence reached $\mathbf{m}^\dagger$; by Proposition 3.1 its Parikh vector $\mathbf{x}^* \geq \mathbf{0}$ satisfies $\mathbf{N}\mathbf{x}^* = \mathbf{b}$, so $\mathbf{y}^\top \mathbf{b} = \mathbf{y}^\top \mathbf{N}\mathbf{x}^* = (\mathbf{N}^\top \mathbf{y})^\top \mathbf{x}^* \leq 0$, contradicting $\mathbf{y}^\top \mathbf{b} > 0$. Hence (b) rules out reachability. Soundness is one-directional by Proposition 3.1: feasibility of the relaxation (a) is necessary but not sufficient for an actual firing sequence, so (a) does not by itself certify conformance — only (b) certifies its negation. $\square$

Corollary 4.2 (The certificate is bounded and verifiable at $O(\kappa)$). *A nonconformance certificate is a single vector $\mathbf{y} \in \mathbb{R}^p$ together with the two inequalities $\mathbf{N}^\top \mathbf{y} \leq \mathbf{0}$ and $\mathbf{y}^\top \mathbf{b} > 0$. Checking it is a matrix product and two sign tests — independent of the length of the trace that produced $\mathbf{m}^\dagger$. Projected to the receipt’s bounded coordinate (a verdict and a localized witness), it is verifiable within a bounded context κ , per the keystone’s Comprehension–Verification Gap.*

Proof. Verification reads $\mathbf{y}$ and $\mathbf{b}$ and evaluates $\mathbf{N}^\top \mathbf{y}$ and $\mathbf{y}^\top \mathbf{b}$; the cost is $O(p \cdot |T|)$ in the net's fixed dimensions, with no dependence on trace length or interior computation. The receipt exposes the Boolean verdict and the witness coordinate, of size $\leq \kappa$. $\square$

Remark (Geometry is the process-layer projection principle). A conformance failure is not reported as “the trace looked wrong”; it is a hyperplane one can recompute. This is the projection principle in linear algebra: an unbounded execution is judged by a bounded certificate whose fidelity is mechanical, not a matter of the verifier comprehending the run.

Chapter 5

Token-Replay Fitness as Normalized Distance

5.1 The replay verifier as a Petri semantics

Token-replay conformance [5] plays a trace against a process model and measures how far it strays from lawful firing. `bcinr-powl`'s `PowlReplayVerifier` is a branchless realization; we give its exact semantics and prove what it characterizes.

Definition 5.1 (Replay state and step). The verifier holds a marking $\mathbf{m}$ (`enabled_tokens`) initialized to the entry token, and accumulators `replayed`, `fitted`, `enabled_not_taken` (bitsets of node identities). A frame carries a one-hot `node_bit`, a preset `required_tokens` = $\mathbf{m}^-$, and a postset `produces_tokens` = $\mathbf{m}^+$. Replaying a frame:

- (1) rejects a `node_bit` that is zero or not a power of two (`UnknownNode`);
- (2) requires $\mathbf{m} \geq \mathbf{m}^-$ (else `TokenNotEnabled`), the branchless check of Proposition 3.2;
- (3) records the enabled-but-unconsumed tokens `enabled_not_taken` $\mid= \mathbf{m} \ \& \ \neg \mathbf{m}^- \ \& \ \neg \text{node_bit}$;
- (4) fires: $\mathbf{m} \leftarrow (\mathbf{m} \ \& \ \neg \mathbf{m}^-) \mid \mathbf{m}^+$; and
- (5) sets the `node_bit` in both `replayed` and `fitted`.

Definition 5.2 (Conformance fitness). On finalization the verifier returns, in Q16.16 fixed point,

$$\varphi = \frac{|\text{fitted}|}{|\text{replayed}|}, \quad \text{precision} = \frac{|\text{replayed}|}{|\text{replayed}| + |\text{enabled_not_taken}|},$$

where $|\cdot|$ is population count and $\varphi := 1$ when the denominator is zero. The value 1.0 is the constant `0x0001_0000`.

Theorem 5.1 (Unit fitness characterizes genuine firing sequences; the fault localizes). *Replaying a sequence of frames against the net either (i) completes with no violation, in which case the sequence is a genuine firing sequence — each transition was enabled at the marking where it fired — and $\varphi = 1$; or (ii) halts at the first frame whose preset is not contained in the current marking, returning `TokenNotEnabled`{`node_id`}, a coordinate-localized witness of the earliest disabled transition.*

Proof. Step (2) of Definition 5.1 admits a frame iff $\mathbf{m} \geq \mathbf{m}^-$, which by Definition 3.1 is exactly the enabling condition; step (4) is the firing rule (Proposition 3.2). By induction on the prefix, if no frame is rejected then every frame fired from a marking at which it was enabled, so the whole sequence is a genuine firing sequence and the intermediate markings are reachable. In that case every replayed frame also sets **fitted** (step 5), so $|\mathbf{fitted}| = |\mathbf{replayed}|$ and $\varphi = 1$. Conversely the guard in step (2) fails at the *first* frame whose required tokens are absent — the verifier returns immediately with that frame’s **node_id** — so a non-firing sequence is rejected at the earliest offending index, which is the localized witness t^* . $\square$

Proposition 5.2 (Fitness and precision are honest population ratios). *φ and precision are ratios of bit-populations of disjointly-maintained bitsets; neither can exceed 1, and $\varphi = 1$ is attained exactly on completed replays (Theorem 5.1). The **enabled_not_taken** accumulator, gathered before consumption in step (3), counts tokens that were live but not used by the firing — the model’s under-fitting — and feeds precision, not fitness; the two metrics therefore measure orthogonal deviations and cannot be traded against each other.*

Proof. Population count is monotone and bounded by the number of node bits, and **fitted** $\subseteq$ **replayed** by construction (step 5 sets both), so $\varphi \leq 1$ with equality iff no frame was replayed-without-fitting — which, in the current step, is every completed replay. **enabled_not_taken** is updated only in step (3) and read only by precision’s denominator, so it is independent of the fitness numerator. $\square$

The lifecycle instance makes this concrete. **replay_lifecycle** on the in-order sequence [Judged, Admitted, Receipted] returns $\varphi = 0x0001_0000$; the out-of-order [Admitted, ...] fails at the Admitted frame, which requires TOK_JUDGED that the entry marking lacks, returning **TokenNotEnabled**{*node_id* : 2}; and skipping **admit** before **receipt** fails at node 3. These are the three tests **lawful_in_order_sequence_has_fitness_one**, **out_of_order_sequence_is_token_not_enabled**, and **skipping_admit_before_receipt_is_token_not_enabled** — Theorem 5.1’s two branches, executed.

Remark (Relation to the keystone’s fractional fitness). The keystone defines $\varphi = 1 - (\text{tokens forced on unenabled tr})$ a fraction in $[0, 1]$. The **praxis** lifecycle verifier is the strict specialization in which a forced-disabled firing is a hard **TokenNotEnabled** rather than a fractional penalty: the “forced consumption” branch is never taken because the verifier refuses to fire a disabled transition at all. Both characterize the same lawful set $\{\varphi = 1\}$; the specialization is more conservative — it localizes the first fault and stops — which is the desired behavior for a lifecycle whose only lawful order is a single line.

Chapter 6

POWL 2.0: Choice Graphs and Partial Orders

6.1 The language

Partially Ordered Workflow Language (POWL) [6] is the model class into which the planner's output is compiled for replay. We define it as the recursive structure the code builds.

Definition 6.1 (POWL model). A *POWL model* over an activity alphabet A is one of:

- (i) an *activity* $a \in A$ (a leaf);
- (ii) a *partial order* $(\{M_1, \dots, M_k\}, \prec)$: a finite set of child POWL models with an irreflexive, acyclic precedence relation $\prec$, executed by respecting $\prec$ and running $\prec$ -incomparable children concurrently; or
- (iii) a *choice graph* $(\{M_1, \dots, M_k\}, E)$: children combined by exclusive choice (and, in POWL 2.0, cyclic/loop edges E), of which the execution takes exactly one live branch.

The three node kinds are `PowlOpKind::Activity`, `PartialOrderGate`, `ChoiceGate` in `powl_bridge`.

Construction 6.1 (Plan to POWL tape; transitive reduction). `temporal_plan_to_powl_tape` converts a `TemporalPlan` into a tape of `PowlOpSpecs`. Each step i becomes an activity with a `pred_mask` initialized to the set of steps $j < i$ that finish before i starts (`prev_end ≤ starti`), and a one-hot `succ_mask = 1 ≪ i`. A second pass performs *transitive reduction*: for each predecessor k of i , the bits of k 's own predecessors are cleared from i 's mask, leaving only *direct* predecessors.

Proposition 6.1 (The reduced masks are the Hasse diagram of the precedence order). *Let $\prec$ be the strict order “ j finishes before i starts” on plan steps. The initialized `pred_mask` of step i is $\{j : j \prec i\}$, and after transitive reduction it is the covering relation of $\prec$: j remains a predecessor of i iff $j \prec i$ and there is no k with $j \prec k \prec i$. The resulting dependency graph is a DAG, and two steps are schedulable concurrently iff they are $\prec$ -incomparable.*

Proof. $\prec$ is irreflexive and transitive (it is induced by the total order on real finish/start times), hence a strict partial order. The first pass sets bit j in i 's mask exactly when $j \prec i$. The reduction clears bit j from i 's mask when some retained predecessor k of i has j in its

predecessors, i.e. $j \prec k \prec i$; what survives is precisely the covering relation (the Hasse diagram) of $\prec$. Acyclicity follows from irreflexivity and transitivity. Incomparable steps have no path between them, so the scheduler may overlap their intervals; comparable steps are ordered by $\prec$. $\square$

Proposition 6.2 (Local marking dimension is bounded by node arity). *In a POWL model, the token marking local to a node ranges over the tokens of that node's immediate children: a partial-order or choice node of arity k has a local marking space of dimension $O(k)$, independent of the total size of the model. Consequently the replay of a node is checked in a working set bounded by its arity, and the global replay is the composition of these bounded-dimension checks.*

Proof. By Definition 6.1 a node's execution semantics reference only its own children's tokens: a partial order gates on the $\prec$ -predecessors among its k children; a choice gates on which of its k branches is live. The token bits touched by one node's firing are therefore contained in an $O(k)$ -dimensional subspace of the marking, regardless of how large the surrounding model is. The bounded-arity constant is the `PowlOpSpec` tape's per-op mask width. $\square$

Proposition 6.2 is the structural reason conformance is cheap: even though the mission's interior may be arbitrarily large, each POWL node is judged in a working set bounded by its arity — the projection principle, localized to the process tree.

Chapter 7

Separability as a Rice Quarantine for Processes

7.1 The separable decomposition

Not every workflow net is a POWL model; the ones that are, are exactly the *separable* ones, and this is a decidable structural property.

Theorem 7.1 (Separable decomposition, after Kourani–van der Aalst). *A sound, separable workflow net admits a recursive decomposition into a POWL model (Definition 6.1) whose every internal node is a partial order (all concurrent / sequential logic) or a choice graph (all exclusive / cyclic logic), and whose leaves are activities. Each node’s local marking geometry has dimension bounded by its arity (Proposition 6.2), independent of the global net size.*

Attribution. This is the decomposition established for POWL by Kourani and van der Aalst and colleagues [6], resting on van der Aalst’s process-mining foundations [7]. We cite it rather than reprove it; the contribution here is the reframing below. $\square$

Proposition 7.2 (Separability is the process-layer Rice quarantine). *Let $\text{sep} : \{\text{workflow nets}\} \rightarrow \{0, 1\}$ be the (decidable) predicate “the net is sound and separable.” Then sep is a legitimate admission obligation g in the sense of the foundations paper: it is total and computable, so admitting a process by sep retracts the space of arbitrary control-flow onto the decidable sub-language of POWL-expressible processes. A separable net is admitted (it decomposes, and its conformance is the bounded-arity replay of Chapter 6); a non-separable net is refused with reason — the reason being the structural obstruction to decomposition.*

Proof. Soundness and separability are structural properties of a finite net, checkable by the decomposition procedure of Theorem 7.1, which either returns a POWL tree or the node at which decomposition fails. Thus sep is total and computable, satisfying the obligation criterion (a syntactic terminating check, never a semantic decision about what the process “means”). Admission maps the net to its POWL decomposition when $\text{sep} = 1$ and to $\perp$ with the obstruction as refusal data otherwise, which is exactly the admission map α specialized to processes. $\square$

Remark (The parallel is exact). The foundations paper retracted the undecidable observation space $\mathcal{O}$ onto a decidable admitted language $\mathcal{O}^*$ because Rice’s theorem forbids deciding

meaning. Here an unwieldy space of arbitrary control-flow is retracted onto the decidable sub-language of separable (POWL) processes because only there is conformance a bounded-arity replay. In both cases: a large, badly-behaved space is replaced by a small decidable sub-language whose membership is mechanically certified, and the boundary certificate — a POWL tree, a fitness value, a Farkas hyperplane — is small. Separability *is* admission, for processes.

Chapter 8

Cost Arbitration and the Makespan Functional

8.1 The cost vector and lexicographic order

When several admitted plans achieve a goal, the router must choose one deterministically. The choice is a total order on a cost vector.

Definition 8.1 (Cost vector). The router's `CostVector` is the tuple

$$\mathbf{c} = (c_0, c_1, c_2, c_3, c_4, c_5) = (\overline{\text{admitted}}, \text{risk}, \text{attention_seconds}, \text{tokens}, \text{latency}, \text{switches}),$$

where $c_0 = \overline{\text{admitted}}$ is the unadmitted indicator ($0 = \text{admitted}$, and `admitted` sorts first), $c_1 = \text{unreceipted_mutation_risk} \in \{0, \dots, 255\}$, $c_2 = \text{human_attention_seconds}$ (the makespan, §8.2), $c_3 = \text{token_cost}$, $c_4 = \text{latency_ms}$, and $c_5 = \text{context_switches} \in \{0, \dots, 255\}$ (distinct capabilities used). The order is lexicographic, cheapest-first, and the refused vector is `refused()` = (unadmitted, 255, ∞ , ∞ , ∞ , 255), the top element.

Theorem 8.1 (Lexicographic order is the infinite-weight limit). *Let the secondary coordinates be bounded, $|c_i| \leq M$ for $i \geq 1$ over the admitted plans (the risk and switch coordinates are ≤ 255 ; the makespan, latency, and token coordinates are bounded by the structural depth and duration constants of Theorem 2.1). Define the scalarization $C_\lambda(\mathbf{c}) = \sum_{i=0}^5 \lambda^{5-i} c_i$ with $\lambda > 1$. Then there is a threshold Λ such that for all $\lambda > \Lambda$,*

$$\mathbf{c} \preceq_{\text{lex}} \mathbf{c}' \iff C_\lambda(\mathbf{c}) \leq C_\lambda(\mathbf{c}').$$

As $\lambda \rightarrow \infty$ the weight λ^5 on the admitted coordinate c_0 diverges relative to all others.

Proof. If $\mathbf{c} \prec_{\text{lex}} \mathbf{c}'$ they first differ at some coordinate p with $c'_p - c_p \geq \epsilon > 0$ (the coordinates are integers or bounded reals with a least positive gap ϵ on the admitted, risk, and switch coordinates; for the continuous coordinates take ϵ the observed resolution). Then

$$C_\lambda(\mathbf{c}') - C_\lambda(\mathbf{c}) \geq \lambda^{5-p} \epsilon - 2M \sum_{i>p} \lambda^{5-i} = \lambda^{5-p} \epsilon - 2M \frac{\lambda^{5-p} - 1}{\lambda - 1},$$

which is positive once $\lambda > \Lambda := 1 + 2M \cdot 6/\epsilon$. Ties in all coordinates map to equality. Hence for $\lambda > \Lambda$ the scalarization order agrees with $\preceq_{\text{lex}}$. The admitted coordinate carries weight λ^5 , dominating the others' λ^{5-i} as $\lambda \rightarrow \infty$. $\square$

Corollary 8.2 (Admitted dominates: no finite cost buys a refused route). *An admitted plan sorts strictly before any refused plan, regardless of its secondary costs: for any finite risk, makespan, token, latency, and switch values, an admitted `CostVector` is $\prec_{\text{lex}}$ the refused top element. Equivalently, the price of unlawfulness is infinite in the limit that defines the order.*

Proof. The lead coordinate c_0 is 0 for admitted and 1 for refused, and by Theorem 8.1 it dominates: any admitted vector differs from the refused element first at c_0 , where it is strictly smaller. Concretely, `CostVector`’s `Ord` places `admitted = true` ahead by reversing the boolean comparison, then breaks ties down the remaining coordinates; the `refused()` constructor sets c_0 to unadmitted and every secondary coordinate to its maximum (∞ for the reals, `u8::MAX/u64::MAX` for the integers), so it is the unique top. This is the test `cost_vector_orders_admitted_before_refused`. $\square$

Remark (The router’s determinism is a receipt). `route_capability_plan` binds the problem text, the plan’s steps, and the cost into a BLAKE3 `route_chain` [9]; identical tasks produce identical chains (`routing_is_deterministic_same_task_same_route`). The arbitration is thus not a heuristic tie-break but a receipted, replayable choice — the cost order is a function, and its value is committed.

8.2 The makespan functional and critical-path structure

The primary continuous cost coordinate c_2 is the makespan, and its structure is what `analyze_schedule` computes over the plan’s POWL tape.

Definition 8.2 (Makespan functional; forward and backward pass). Given the precedence DAG of Construction 6.1 with per-op durations d_i and release times, the *forward pass* computes earliest starts and finishes

$$\text{es}_i = \max\left(r_i, \max_{j \prec i} \text{ef}_j\right), \quad \text{ef}_i = \text{es}_i + d_i,$$

the *makespan* is $T = \max_i \text{ef}_i$, and the *backward pass* computes latest starts $\text{ls}_i = \text{lf}_i - d_i$ with $\text{lf}_i = \min_{i \prec k} \text{ls}_k$ (or T at sinks). The *slack* of op i is $\text{ls}_i - \text{es}_i$, and the *critical path* is the set of zero-slack ops.

Proposition 8.3 (Makespan is the longest path; the critical path is zero-slack). *The makespan T equals the length of the longest ($\prec$ -)path in the precedence DAG weighted by durations, and an op has zero slack iff it lies on some longest path. Delaying a zero-slack op delays the makespan; delaying an op by less than its slack does not.*

Proof. The forward recursion $\text{ef}_i = \text{es}_i + d_i$ with $\text{es}_i = \max_{j \prec i} \text{ef}_j$ is the standard longest-path dynamic program over a DAG (topologically ordered here because op i ’s predecessors have indices $< i$), so ef_i is the longest weighted path ending at i and $T = \max_i \text{ef}_i$ is the overall longest path. The backward pass gives the latest each op may start without increasing T ; slack $\text{ls}_i - \text{es}_i = 0$ iff i cannot move, i.e. it lies on a longest path. This is exactly `forward_pass`, `backward_pass`, and the `critical_path_mask` (bit i set iff slack $< 10^{-9}$) of `ScheduleAnalysis64`. $\square$

Proposition 8.4 (Capacity sensitivity is a finite difference of the found schedule). *Let $T(\phi = c)$ be the makespan of the schedule the planner finds when fluent ϕ ’s initial capacity is c . The*

capacity delta reports $(T(c-1), T(c), T(c+1))$, and ϕ is binding iff $T(c-1) \neq T(c)$ (or the problem is infeasible at $c-1$). This is a finite-difference sensitivity of the specific schedule the greedy planner produced, not a dual shadow price of an optimal scheduler.

Proof. `replan_with_perturbed_capacity` re-solves `find_temporal_plan` with ϕ 's initial value moved by ± 1 (clamped at 0) via `find_temporal_plan_with_fn_overrides`, and reads off the resulting makespan; `binding_resource_mask` sets bit i iff the -1 perturbation changed the makespan or made the problem infeasible. Because `find_temporal_plan` is a greedy forward-chaining planner (its own module comment disclaims optimality), the delta is the response of that found schedule, which is the honest claim the code makes — and no more. Infeasibility at $c-1$ is itself evidence the resource binds, per Proposition 2.3. $\square$

Remark (The functional closes the loop). The makespan feeds back into arbitration: $c_2 = \text{human_attention_seconds} = \text{analysis.makespan}(\text{CostVector::from_analysis})$, so the primary continuous cost the lexicographic order minimizes is the longest-path length of Proposition 8.3. Attention conservation (Proposition 2.2) says the total attention spent is schedule-invariant; the makespan functional says only its *completion time* is what arbitration optimizes — exactly the keystone's "reordering redistributes when attention is spent, not how much."

Chapter 9

Conclusion

We gave the geometry beneath two of the enforced equation’s invariants. Mission manufacture is bounded search over a finite ground net (Theorem 2.1); the state it searches is a marking with a numeric fluent valuation, and the attention fluent, borrowed and returned, obeys a balance law whose nonnegativity is feasibility (Propositions 2.2–2.3). The reachable markings lie in a translated cone (Proposition 3.1), the safe-net specialization of which is the branchless firing rule the receipt path runs (Proposition 3.2). Conformance is membership in that cone, and a nonconformant trace is separated by a Farkas hyperplane — a bounded, sound certificate verifiable at $O(\kappa)$ (Theorem 4.1, Corollary 4.2). Token-replay fitness equals 1 exactly on genuine firing sequences, with the first disabled transition a localized witness (Theorem 5.1), and it is an honest population ratio (Proposition 5.2). POWL 2.0 gives the recursive partial orders and choice graphs (Definition 6.1); the plan-to-tape transitive reduction is the Hasse diagram of the precedence order (Proposition 6.1), and each node is judged in a working set bounded by its arity (Proposition 6.2). The Kourani–van der Aalst separable decomposition (Theorem 7.1) is the Rice quarantine for processes: separable is admitted, non-separable is refused with reason (Proposition 7.2). And cost arbitration is the infinite-weight limit in which the admitted coordinate dominates (Theorem 8.1, Corollary 8.2), over a makespan functional that is the longest path of the precedence DAG (Proposition 8.3).

The standard was never comprehension of the mission’s interior. It is a bounded search, a membership test with a small certificate, and a fitness value of one — each guaranteed by mechanism, each small enough to hold.

Conformance is distance to a cone; the certificate is a hyperplane.

Appendix A

Notation

Symbol	Meaning
$\mathbf{m}, \mathbf{m}_0$	marking; initial marking (vectors in $\mathbb{Z}_{\geq 0}^p$)
$\mathbf{m}_t^-, \mathbf{m}_t^+$	preset (consumed) / postset (produced) of transition t
$\mathbf{N}, \mathbf{x}$	incidence matrix; Parikh (firing-count) vector
$\mathcal{P}, \text{cone}(\mathbf{N})$	marking polytope; reachability cone
$\mathbf{y}$	Farkas separating-hyperplane normal (nonconformance certificate)
φ	token-replay fitness $\in [0, 1]$ (Q16.16; 1.0 = 0x0001.0000)
$f_\phi(t)$	free level of numeric fluent ϕ at time t
$T, \text{es}_i, \text{ls}_i$	makespan; earliest / latest start of op i
$\mathbf{c}, \preceq_{\text{lex}}$	cost vector; lexicographic cost order
$\mu, \mathcal{O}^*, \perp, \kappa$	manufacturing morphism; admitted space; refusal; verifier context

Appendix B

Code Correspondence

Every object above is anchored to the `bcinr-pddl` / `bcinr-powl` / `praxis` source.

Mathematical object	Code artifact	Location
Grounded temporal problem	<code>GroundTemporalProblem::build</code>	<code>bcinr-pddl/src/ground.rs</code>
Bounded grounding	<code>PDDL8_MAX_GROUND</code> , <code>BoundExceeded</code>	<code>bcinr-pddl/src/ground.rs</code>
Forward search (manufacture)	<code>GroundProblem::find_plan</code> (BFS)	<code>bcinr-pddl/src/ground.rs</code>
Numeric fluent update	<code>eval_numeric</code> , <code>apply_numeric_effect</code>	<code>bcinr-pddl/src/ground.rs</code>
Attention fluent	<code>(:functions (attention)) domain</code>	<code>bcinr-pddl/src/capability.rs</code>
Firing rule (safe net)	<code>PowlReplayVerifier::replay_frame</code>	<code>bcinr-powl-receipt/src/replay.rs</code>
Fitness / precision	<code>ConformanceMetrics</code> , <code>finalize</code>	<code>bcinr-powl-receipt/src/replay.rs</code>
Lifecycle token net	<code>TOK_*</code> , <code>replay_lifecycle</code>	<code>praxis-core/src/replay_adapter.rs</code>
POWL tape / reduction	<code>temporal_plan_to_powl_tape</code>	<code>bcinr-pddl/src/powl_bridge.rs</code>
Cost vector / order	<code>CostVector</code> , <code>impl Ord</code>	<code>bcinr-pddl/src/capability.rs</code>
Route receipt	<code>route_capability_plan</code>	<code>bcinr-pddl/src/capability.rs</code>
Makespan / critical path	<code>analyze_schedule</code> , <code>ScheduleAnalysis64</code>	<code>bcinr-pddl/src/schedule_analysis.rs</code>
Capacity sensitivity	<code>replan_with_perturbed_capacity</code>	<code>bcinr-pddl/src/schedule_analysis.rs</code>
CLI surface	<code>plan solve/analyze/route/execute</code>	<code>praxis/src/verbs/plan.rs</code>

Bibliography

- [1] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.
- [2] M. Fox and D. Long, *PDDL2.1: An extension to PDDL for expressing temporal planning domains*, Journal of Artificial Intelligence Research **20** (2003), 61–124.
- [3] T. Murata, *Petri nets: Properties, analysis and applications*, Proceedings of the IEEE **77**(4) (1989), 541–580.
- [4] A. Schrijver, *Theory of Linear and Integer Programming*, Wiley, 1986.
- [5] A. Rozinat and W. M. P. van der Aalst, *Conformance checking of processes based on monitoring real behavior*, Information Systems **33**(1) (2008), 64–95.
- [6] H. Kourani and S. J. van Zelst, *POWL: Partially Ordered Workflow Language*, in Business Process Management (BPM 2023), Lecture Notes in Computer Science **14159**, Springer, 2023, 92–108.
- [7] W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd ed., Springer, 2016.
- [8] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.
- [9] J. O’Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O’Hearn, *BLAKE3: One function, fast everywhere*, 2020.